Git, Test Configuration and Cross-Browser Testing

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–13 concepts (Locators, assertions, fixtures, POM, and API testing).
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Prepare your Playwright automation framework for enterprise team collaboration. Master Git pull request reviews, production playwright.config.js multi-browser matrixes, and targeted CLI execution.

Day 14 Learning Objectives

'Master Git workflows for QA engineers: branching, commits, pull requests, merge conflicts, and .gitignore hygiene.
'Configure playwright.config.js for enterprise environments: baseURL, timeouts, retries, workers, and reporters.
'Configure multi-browser test execution across Chromium, Firefox, and WebKit (Safari engine).
'Configure mobile viewport device emulation (e.g. Pixel 7, iPhone 14) using Playwright devices.
'Execute targeted CLI commands: filtering tests by title grep, targeting specific files, and selecting browser projects.
'Eliminate committed secrets and hardcoded credentials from configuration files.
'Review automation pull requests with an engineering checklist for reliability and maintainability.

In modern engineering teams, automated tests live in Git version control alongside application source code.

Core Git Workflow

1. **Branching: Never push directly to `main`. Create feature branches: `git checkout -b feature/cart-automation`.**
2. **Atomic Commits: Make small, logical commits with conventional commit messages:**
- `feat(checkout): add payment method validation tests`
- `fix(auth): update login button locator for redesigned navbar`
3. **Pull Requests (PRs): Submit a PR for peer review before merging. CI runs your tests automatically on the PR branch!**
4. **Merge Conflicts: Occur when two teammates edit the same line of a Page Object or test file. Resolve by rebasing or merging `main`.**

What to Include in `.gitignore`

```text
# Crucial ignore rules for Playwright projects:
node_modules/
.env
.env.local
playwright/.auth/
test-results/
playwright-report/
blob-report/
```

Git Branch & PR Review Flow (Standard team workflow for authoring automation scripts.)

```
git checkout -b test/day-14-config
git add .
git commit -m "test(config): configure multi-browser matrix for chromium, firefox, webkit"
git push origin test/day-14-config
```

`playwright.config.js` is the control center for your entire automation framework.

Key Configuration Properties

```javascript
// playwright.config.js
const { defineConfig, devices } = require('@playwright/test');
module.exports = defineConfig({
  testDir: './tests',
  fullyParallel: true,              // Run tests across files in parallel
  forbidOnly: !!process.env.CI,     // Fail CI if test.only() was accidentally committed!
  retries: process.env.CI ? 2 : 0,  // Retries in CI, fail-fast locally
  workers: process.env.CI ? 2 : undefined, // Parallel workers
  timeout: 30000,                   // 30s max duration per test
```

```
  expect: {
    timeout: 5000,                  // 5s max duration for web-first assertions
  },
  reporter: [
    ['list'],
    ['html', { open: 'never' }],
  ],
  use: {
    baseURL: process.env.BASE_URL || 'https://staging.rootaccess.org',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
});
```

The `forbidOnly` Rule
If a developer leaves `test.only()` in a file, only that single test runs in CI, silently skipping the other 499 tests! `forbidOnly: !!process.env.CI` causes CI to immediately fail with a build error, preventing unverified code merges.

Configuring Environment-Specific baseURL (Dynamically selecting test environments.)

```
const environments = {
  staging: 'https://staging.rootaccess.org',
  qa: 'https://qa.rootaccess.org',
  prod: 'https://rootaccess.org',
};

const baseURL = environments[process.env.ENV || 'staging'];
```

Modern web applications must work seamlessly across diverse browser rendering engines and mobile viewports.

The 3 Core Browser Engines in Playwright
Playwright bundles and executes tests against true browser rendering engines:
1. **Chromium: Powers Google Chrome, Microsoft Edge, Brave, and Opera.**
2. **Firefox (Gecko): Powers Mozilla Firefox.**
3. **WebKit: Powers Apple Safari on macOS and all browsers on iOS (iPhones and iPads).**

Configuring Multi-Browser Projects
```javascript
// playwright.config.js
const { devices } = require('@playwright/test');

```
module.exports = {
  projects: [
    // Desktop Browsers
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
    // Mobile Emulation
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 7'] },
    },
    {
      name: 'Mobile Safari',
      use: { ...devices['iPhone 14'] },
    },
```

```
  ],
};
```

Why a Test Passing in Chromium Might Fail in WebKit
- CSS rendering differences (e.g. flexbox wrapping, date input pickers).
- Touch event requirements on mobile viewports.
- Cookie policy differences (WebKit has stricter third-party cookie blocking).

Running Tests on a Specific Browser Engine (Executing tests exclusively on WebKit via CLI.)

```
npx playwright test --project=webkit
```

During development, you should never wait for the entire 500-test suite to run just to verify a 1-line change.

Essential CLI Commands
```bash
 # 1. Run a specific test file:
npx playwright test tests/auth/login.spec.js
2. Filter tests by title regex (-g or --grep):
npx playwright test -g "invalid password"

3. Inverse filter (--grep-invert):
npx playwright test --grep-invert "@slow"

4. Target a specific project:
npx playwright test --project=chromium

5. Run headed (watch in browser):
npx playwright test --headed

6. Interactive step-through debugging:
npx playwright test --debug

7. Modern visual UI Mode:
npx playwright test --ui

8. Specify number of parallel workers:
npx playwright test --workers=4
```

**Tagging Tests with `@smoke`, `@regression`, `@api`**
**Incorporate tags into test titles:**
**```javascript**
**test('fast login verification @smoke @auth', async ({ page }) => { ... });**
**```**
**Now run only smoke tests:**
**```bash**
**npx playwright test --grep "@smoke"**
**```**

**Running a Targeted Smoke Test Suite (Targeting smoke tests on Chromium in headed mode.)**

```
npx playwright test -g "@smoke" --project=chromium --headed
```

requirements.
```

Assignment Starter Code (Day 14)

```
// playwright.config.js - Enterprise Production Configuration
const { defineConfig, devices } = require('@playwright/test');

/**
 * Clean, production-hardened Playwright configuration:
 * 1. Multi-browser support (Chromium, Firefox, WebKit) + Mobile Emulation
 * 2. Environment-based baseURL (Staging vs Production)
 * 3. Sensible timeout tiers and retries
 * 4. forbidOnly enabled in CI
 * 5. Diagnostic reporting with Trace Viewer on failure
 * 6. Zero hardcoded credentials or committed secrets
 */
module.exports = defineConfig({
  testDir: './tests',
  fullyParallel: true,

  // Fail CI builds immediately if test.only was accidentally committed!
  forbidOnly: !!process.env.CI,

  // Fail-fast locally (0 retries), absorb transient blips in CI (2 retries)
  retries: process.env.CI ? 2 : 0,
```

DAY 14 INTERVIEW PREPARATION QUESTION BANK

Q1. What is the purpose of playwright.config.js, and what are the most critical settings for a production CI environment?

[Difficulty: Intermediate] Evaluation Focus: Configuration architecture and production stability.

Model Answer: playwright.config.js is the central configuration file defining how Playwright discovers, organizes, and executes tests. For production CI environments, the most critical settings include: `forbidOnly: !!process.env.CI` (to fail the build if test.only was committed), `retries: process.env.CI ? 2 : 0` (to absorb intermittent network hiccups while failing fast locally), `fullyParallel: true` (to maximize worker concurrency), `baseURL` (to decouple test scripts from environment domains), `reporter: [["html", { open: "never" }], ["list"]]`, and `trace: "on-first-retry"` or `"retain-on-failure"` to capture diagnostic artifacts on failure.

Follow-Up Question: "How does configuring workers: 2 in CI prevent resource exhaustion on limited virtual runners?"

Q2. How does Playwright support cross-browser testing across Chromium, Firefox, and WebKit?

[Difficulty: Intermediate] Evaluation Focus: Browser engine architecture and cross-browser coverage.

Model Answer: Playwright bundles binaries for the actual rendering engines: Chromium (Chrome, Edge), Firefox (Gecko), and WebKit (Safari). In `playwright.config.js`, we define separate projects inside the `projects` array for each engine using built-in presets from `@playwright/test` devices (e.g. `Desktop Chrome`, `Desktop Firefox`, `Desktop Safari`). When executed, Playwright runs the exact same test scripts against all configured projects in parallel, ensuring cross-browser rendering and behavior consistency without requiring third-party cloud grids.

Follow-Up Question: "Why is WebKit on Linux useful for teams that develop on Windows or Linux?"

Q3. How do you structure test tags in Playwright to run different suites (e.g. Smoke vs Regression)?

[Difficulty: Beginner] Evaluation Focus: Test selection, naming conventions, and CLI filtering.

Model Answer: We add tag keywords like `@smoke`, `@regression`, or `@critical` directly into test titles (e.g. `test("user checkout flow @smoke @critical", async ({ page }) => { ... })`). In CI pipelines or terminal commands, we use the `--grep` (`-g`) flag to run specific subsets (e.g. `npx playwright test -g "@smoke"`). We can also use `--grep-invert` to exclude specific suites (e.g. `npx playwright test --grep-invert "@slow"`).

Follow-Up Question: "Can tags also be configured at the test.describe() level?"

Q4. What checklist do you use when reviewing an automation pull request submitted by a teammate?

[Difficulty: Advanced] Evaluation Focus: Code review standards, quality gates, and maintainability.

Model Answer: My review checklist includes: 1) Are locators accessible and resilient (`getByRole`, `getByLabel`) rather than brittle XPaths or auto-generated CSS classes? 2) Are assertions web-first (`await expect(...)`) with no arbitrary sleeps (`page.waitForTimeout`)? 3) Are tests independent (no shared mutable state between tests)? 4) Are test credentials loaded securely via `process.env` with zero hardcoded passwords? 5) Does the test clean up database entities or use collision-free unique data? 6) Did they remove any `test.only` or `page.pause` debugging statements? 7) Is reusable logic encapsulated in Page Objects?

Follow-Up Question: "How would you handle a PR where a teammate increased a test timeout to 60 seconds to fix flakiness?"

Q5. How does mobile device emulation in Playwright work, and how does it differ from testing on real physical devices?

[Difficulty: Intermediate] Evaluation Focus: Emulation capabilities, viewports, and testing boundaries.

Model Answer: Playwright provides mobile device emulation using pre-configured presets in `devices` (e.g. `devices["Pixel 7"]` or `devices["iPhone 14"]`). These presets configure the browser context's viewport dimensions, deviceScaleFactor (pixel ratio), isMobile flag, touch event support, and user-agent string. It runs on the desktop Chromium or WebKit engine. While this is fast, reliable, and catches 90% of responsive UI and touch event defects, it does not execute on real mobile hardware or native mobile OS kernels, so device-specific hardware bugs (camera, thermal throttling) still require real device testing.

Follow-Up Question: "Why is touch support emulation important when testing mobile viewports?"

1. **What does the forbidOnly: !!process.env.CI option do in playwright.config.js?**
Key Concept: forbidOnly fails the build if a developer accidentally committed test.only(), preventing accidental skips of the entire test suite in CI.

2. **Which browser rendering engine is used by Playwright to simulate Apple Safari?**
Key Concept: WebKit is the open-source browser engine developed by Apple that powers Safari on macOS and all iOS browsers.

3. **Which CLI flag filters tests by title using a regular expression?**
Key Concept: -g (or --grep) filters tests whose titles match the provided string or regular expression.

4. **How do you target only the Firefox project when running Playwright from the command line?**
Key Concept: --project=<name> restricts test execution to the specified project defined in playwright.config.js.

5. **What does devices["iPhone 14"] provide when added to a project's use configuration?**
Key Concept: Playwright device descriptors bundle standard mobile parameters (viewport size, pixel ratio, touch support, user-agent) for responsive testing.

6. **Why should retries be set to 0 during local development and 1 or 2 in CI?**
Key Concept: Failing fast locally speeds up debugging; in CI, 1–2 retries prevent intermittent external network blips from breaking deployment pipelines.

7. **What is the recommended Git branching strategy for automation engineers working in an Agile team?**
Key Concept: Feature branches with atomic commits and Pull Request reviews ensure code quality, prevent regressions, and trigger automated CI test runs.

8. **What is the purpose of the baseURL option in playwright.config.js?**
Key Concept: baseURL prepends the target server URL to relative paths, enabling the exact same test suite to execute against Staging, QA, or Production.

9. **Which file should be excluded from Git to prevent exposing credentials and environment secrets?**
Key Concept: .env files containing sensitive passwords and API keys must be listed in .gitignore and never committed to version control.

10. **How do you invert a grep filter in Playwright to run all tests EXCEPT those tagged with @slow?**
Key Concept: --grep-invert excludes tests matching the specified pattern, allowing fast test execution by skipping long-running or quarantined tests.